

# Multi-AS Network Simulation in ns-3: BGP-like Routing, Link Failure Recovery, and Performance Analysis

Salih Sefer — 820230313

May 2026

## Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                                          | <b>3</b>  |
| <b>2</b> | <b>Topology Description</b>                              | <b>4</b>  |
| 2.1      | Autonomous System Overview . . . . .                     | 4         |
| 2.2      | IP Addressing Plan . . . . .                             | 4         |
| <b>3</b> | <b>Routing Structure</b>                                 | <b>6</b>  |
| 3.1      | Intra-AS: OSPF-like Shortest-Path Routing . . . . .      | 6         |
| 3.2      | Inter-AS: BGP-like Policy Routing . . . . .              | 6         |
| 3.3      | Link Failure and Recovery Workflow . . . . .             | 7         |
| <b>4</b> | <b>Experimental Setup</b>                                | <b>9</b>  |
| 4.1      | Scenario Matrix . . . . .                                | 9         |
| 4.2      | Traffic Generation . . . . .                             | 9         |
| 4.3      | Failure and Recovery Timeline . . . . .                  | 9         |
| 4.4      | Simulation Parameters . . . . .                          | 10        |
| <b>5</b> | <b>Results</b>                                           | <b>11</b> |
| 5.1      | Summary Statistics . . . . .                             | 11        |
| 5.2      | Performance Plots . . . . .                              | 11        |
| <b>6</b> | <b>Analysis and Interpretation</b>                       | <b>15</b> |
| 6.1      | Effect of Node Count on Network Performance . . . . .    | 15        |
| 6.2      | Balanced vs. Unbalanced Distribution Impact . . . . .    | 15        |
| 6.3      | Topology Change Response: Convergence Analysis . . . . . | 16        |
| 6.4      | Observable Bottlenecks: AS2 as Transit Hub . . . . .     | 16        |
| <b>7</b> | <b>Reproducibility</b>                                   | <b>17</b> |
| 7.1      | Environment Setup . . . . .                              | 17        |
| 7.2      | Running the Simulations . . . . .                        | 17        |
| 7.3      | Generating Analysis Outputs . . . . .                    | 17        |
| 7.4      | Visualising with NetAnim . . . . .                       | 18        |
| <b>8</b> | <b>Conclusion</b>                                        | <b>19</b> |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>9</b>  | <b>Appendix A: Repository Directory Structure</b> | <b>20</b> |
| <b>10</b> | <b>Appendix B: Command-Line Flags</b>             | <b>20</b> |
| <b>11</b> | <b>Appendix C: RNG Seed and Run Configuration</b> | <b>21</b> |

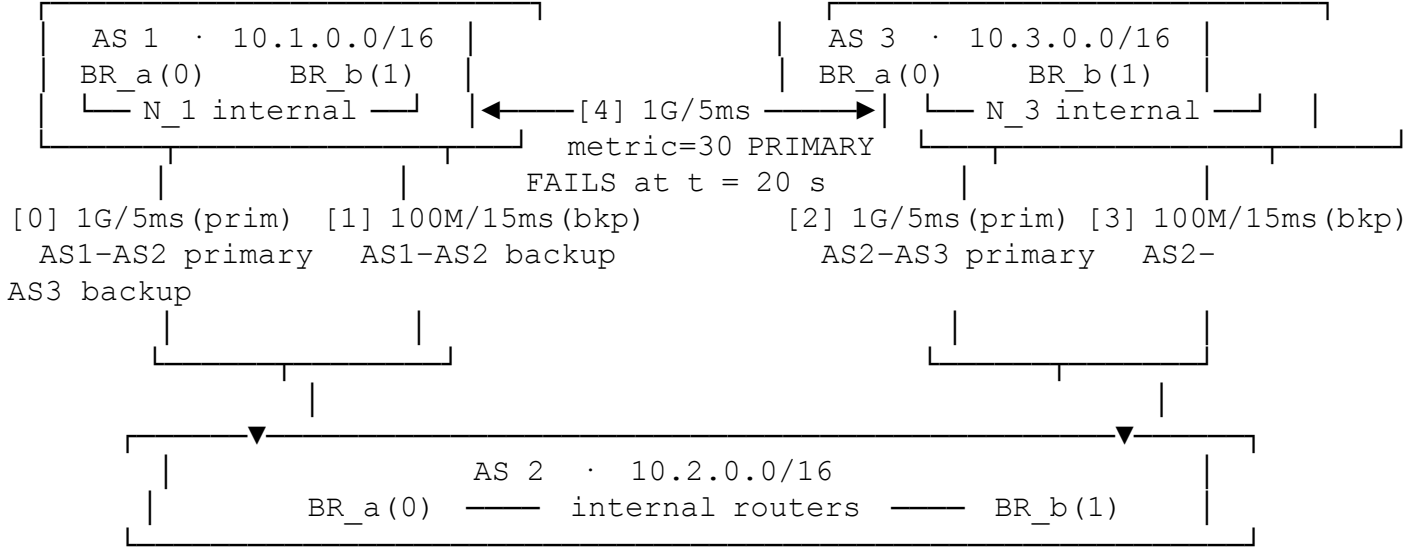
# **1 Abstract**

This report presents a parametric multi-autonomous-system (Multi-AS) network simulation built on ns-3 42. Three interconnected autonomous systems are modelled with randomised intra-AS topologies, BGP-like inter-AS policy routing, and a timed link-failure/recovery scenario. Six scenarios are evaluated across two node-count distribution strategies (balanced and unbalanced) at three scale points (20, 50, and 100 nodes), each replicated three times with distinct RNG seeds. Per-flow metrics — end-to-end delay, throughput, and packet loss — together with the BGP convergence time following a primary-link failure are collected via ns-3 FlowMonitor and aggregated into a comparative analysis. Results show that network scale and distribution asymmetry have measurable but bounded effects on delay and throughput, while convergence time is determined primarily by the simulated BGP hold-timer delay of 2–5 s.

## 2 Topology Description

### 2.1 Autonomous System Overview

The simulation models three ASes interconnected by five point-to-point links. Each AS contains two border routers (BR\_a and BR\_b at node indices 0 and 1 within the AS) plus a variable number of internal routers built using a randomised two-group spanning tree that guarantees two vertex-disjoint paths between every BR pair.



**Figure 1.** Inter-AS topology. The dashed link [4] (AS1.BR\_a ↔ AS3.BR\_a, metric = 30) is the direct path that fails at t = 20 s; after the simulated BGP hold-timer (2–5 s) traffic reroutes via AS 2 using the pre-installed metric = 80 backup routes.

The five inter-AS links are ordered and referred to by index throughout the codebase:

| Idx | Endpoints           | Bandwidth | Delay | Role                   |
|-----|---------------------|-----------|-------|------------------------|
| 0   | AS1.BR_a ↔ AS2.BR_a | 1 Gbps    | 5 ms  | AS1–AS2 primary        |
| 1   | AS1.BR_b ↔ AS2.BR_b | 100 Mbps  | 15 ms | AS1–AS2 backup         |
| 2   | AS2.BR_a ↔ AS3.BR_a | 1 Gbps    | 5 ms  | AS2–AS3 primary        |
| 3   | AS2.BR_b ↔ AS3.BR_b | 100 Mbps  | 15 ms | AS2–AS3 backup         |
| 4   | AS1.BR_a ↔ AS3.BR_a | 1 Gbps    | 5 ms  | AS1–AS3 direct (fails) |

**Table 1.** Inter-AS link parameters. Intra-AS links use 100 Mbps / 2 ms uniformly.

### 2.2 IP Addressing Plan

Each AS owns a private /16 block for its intra-AS /30 subnets. The five inter-AS point-to-point links share a sequential 172.16.1.0/24 address space, advancing by 4 for each new link.

| Block       | Assignment         | Subnet per link                             |
|-------------|--------------------|---------------------------------------------|
| 10.1.0.0/16 | AS1 intra-AS links | /30, allocated by <code>NewNetwork()</code> |

| Block          | Assignment           | Subnet per link                |
|----------------|----------------------|--------------------------------|
| 10.2.0.0/16    | AS2 intra-AS links   | /30, allocated by NewNetwork() |
| 10.3.0.0/16    | AS3 intra-AS links   | /30, allocated by NewNetwork() |
| 172.16.1.0/24+ | Inter-AS links (all) | /30, sequential                |

**Table 2.** IP addressing plan.

Specific inter-AS host addresses (first/second usable address in each /30):

| Link | Subnet         | AS-left IP             | AS-right IP            |
|------|----------------|------------------------|------------------------|
| 0    | 172.16.1.0/30  | 172.16.1.1 (AS1.BR_a)  | 172.16.1.2 (AS2.BR_a)  |
| 1    | 172.16.1.4/30  | 172.16.1.5 (AS1.BR_b)  | 172.16.1.6 (AS2.BR_b)  |
| 2    | 172.16.1.8/30  | 172.16.1.9 (AS2.BR_a)  | 172.16.1.10 (AS3.BR_a) |
| 3    | 172.16.1.12/30 | 172.16.1.13 (AS2.BR_b) | 172.16.1.14 (AS3.BR_b) |
| 4    | 172.16.1.16/30 | 172.16.1.17 (AS1.BR_a) | 172.16.1.18 (AS3.BR_a) |

**Table 3.** Inter-AS /30 host addresses.

### 3 Routing Structure

#### 3.1 Intra-AS: OSPF-like Shortest-Path Routing

All nodes within each AS run ns-3's `Ipv4GlobalRouting`, which computes Dijkstra-based shortest-path forwarding tables over the entire simulated topology. This is installed via `Ipv4GlobalRoutingHelper::PopulateRoutingTables()` after the full node/link graph is built, analogous to an OSPF domain-wide link-state database synchronisation and SPF computation.

`Ipv4ListRouting` is installed on every node with a priority stack:

```
Priority 10 → Ipv4StaticRouting    (border-router policy routes only)
Priority 5  → Ipv4GlobalRouting    (shortest-path for everything else)
```

For non-BR nodes, the static routing table is empty; all lookups fall through to global routing. For BR nodes, the static routes covering remote AS prefixes (/16 aggregates) are matched first; intra-AS destinations fall through to global routing.

**Semantic note.** `Ipv4GlobalRouting` is not OSPF: it does not exchange Link-State Advertisements, does not use configurable per-link costs beyond a unit default, and the SPF runs centrally in simulation time rather than in a distributed protocol. The analogy is structural (link-state topology, loop-free forwarding) rather than protocol-level.

#### 3.2 Inter-AS: BGP-like Policy Routing

Six border routers carry static /16 policy routes installed at  $t = 0$ . The routing intent is modelled after BGP's path-preference mechanism via explicit Dijkstra-metric values: **lower metric = preferred path**.

| BR       | Destination | Next-hop | Via link | Metric    | Role                               |
|----------|-------------|----------|----------|-----------|------------------------------------|
| AS1.BR_a | 0.2.0.0/16  | AS2.BR_a | link[0]  | 50        | to AS2<br>primary                  |
| AS1.BR_a | 0.3.0.0/16  | AS3.BR_a | link[4]  | <b>30</b> | to AS3<br>direct ←<br><b>fails</b> |
| AS1.BR_a | 0.3.0.0/16  | AS2.BR_a | link[0]  | 80        | to AS3<br>via AS2<br>(backup)      |
| AS1.BR_b | 0.2.0.0/16  | AS2.BR_b | link[1]  | 50        | to AS2 via<br>backup               |
| AS1.BR_b | 0.3.0.0/16  | AS2.BR_b | link[1]  | 80        | to AS3<br>via AS2<br>backup        |
| AS2.BR_a | 0.1.0.0/16  | AS1.BR_a | link[0]  | 50        | return to<br>AS1                   |
| AS2.BR_a | 0.3.0.0/16  | AS3.BR_a | link[2]  | 50        | to AS3<br>primary                  |
| AS2.BR_b | 0.1.0.0/16  | AS1.BR_b | link[1]  | 50        | return to<br>AS1                   |

| BR       | Destination | Next-hop | Via link | Metric    | Role                               |
|----------|-------------|----------|----------|-----------|------------------------------------|
| AS2.BR_b | 0.3.0.0/16  | AS3.BR_b | link[3]  | 50        | to AS3<br>backup                   |
| AS3.BR_a | 0.2.0.0/16  | AS2.BR_a | link[2]  | 50        | to AS2<br>primary                  |
| AS3.BR_a | 0.1.0.0/16  | AS1.BR_a | link[4]  | <b>30</b> | to AS1<br>direct ←<br><b>fails</b> |
| AS3.BR_a | 0.1.0.0/16  | AS2.BR_a | link[2]  | 80        | to AS1<br>via AS2<br>(backup)      |
| AS3.BR_b | 0.2.0.0/16  | AS2.BR_b | link[3]  | 50        | to AS2<br>backup                   |
| AS3.BR_b | 0.1.0.0/16  | AS2.BR_b | link[3]  | 80        | to AS1<br>via AS2<br>backup        |

**Table 4.** Border-router policy routes. Ipv4StaticRouting selects the lowest-metric matching entry; higher-metric entries serve as pre-installed backups.

**Semantic note.** This is not full BGP: there is no TCP session, no OPEN/UPDATE/KEEPALIVE exchange, no route-reflector hierarchy, and no AS\_PATH attribute. The simulation captures BGP’s policy-routing semantics (preferred vs. backup paths, hold-timer delay before route withdrawal) while staying within ns-3’s routing API.

### 3.3 Link Failure and Recovery Workflow

The failure sequence for the AS1\_AS3\_PRIMARY link is:

```

t = failureTime (default 20 s)
→ BgpLikeRouter::TriggerFailure()
    1. Ipv4::SetDown() on both endpoints of link[4]
       (AS1.BR_a interface and AS3.BR_a interface go DOWN)
    2. MetricsCollector::MarkFailureEvent() – convergence timer starts
    3. Schedule RecomputeBgpTables() after Uniform[2, 5] s delay

t = failureTime + Uniform[2, 5] s
→ BgpLikeRouter::RecomputeBgpTables()
    1. Remove all static routes whose next-hop is reachable only via link[4]
       (metric=30 routes on AS1.BR_a and AS3.BR_a withdrawn)
    2. Ipv4GlobalRoutingHelper::RecomputeRoutingTables()
       (global SPF updated for non-BR nodes)
    3. MetricsCollector::MarkConvergenceEvent() – convergence time recorded
→ Surviving metric=80 routes on AS1.BR_a and AS3.BR_a now best match
→ Traffic path: AS1.BR_a → link[0] → AS2.BR_a → link[2] → AS3.BR_a

t = failureTime + 30 s (= 50 s by default)
→ BgpLikeRouter::TriggerRecovery()

```

1. `Ipv4::SetUp()` restores `link[4]`
2. `Metric=30` routes re-added to BR static tables
3. `RecomputeRoutingTables()` – direct path restored

The BGP hold-timer analogue (`Uniform[2, 5]` s) simulates the real-world delay between link failure detection and route withdrawal propagation in a BGP-speaking network.



## 4 Experimental Setup

### 4.1 Scenario Matrix

Node counts are distributed across the three ASes according to the selected strategy.

| Scenario | N   | Distribution | N_1 (AS1) | N_2 (AS2) | N_3 (AS3) | Flows |
|----------|-----|--------------|-----------|-----------|-----------|-------|
| S1       | 20  | balanced     | 7         | 7         | 6         | 6     |
| S2       | 20  | unbalanced   | 4         | 6         | 10        | 6     |
| S3       | 50  | balanced     | 17        | 17        | 16        | 9     |
| S4       | 50  | unbalanced   | 10        | 15        | 25        | 9     |
| S5       | 100 | balanced     | 34        | 34        | 32        | 14    |
| S6       | 100 | unbalanced   | 20        | 30        | 50        | 14    |

**Table 5.** Scenario matrix.  $N_i$  computed by: balanced =  $\text{ceil}(N/3) / \text{ceil}(N/3) / \text{remainder}$ ; unbalanced =  $\text{round}(N \times 0.2) / \text{round}(N \times 0.3) / \text{remainder}$ . Flows =  $4 + \text{floor}(N/10)$ .

Each scenario is executed three times (run IDs 1–3) with seeds `BASE_SEED + run_id` (default `BASE_SEED = 1`), yielding 18 independent simulation runs in total.

### 4.2 Traffic Generation

UDP traffic is modelled with ns-3 `OnOffHelper` (constant-on) and `PacketSinkHelper` pairs. Sources and sinks are placed on non-BR nodes ( $\text{index} \geq 2$  within each AS) chosen uniformly at random; BRs are used only if an AS has fewer than three nodes.

| Flow set        | Directions                                    | Count                |
|-----------------|-----------------------------------------------|----------------------|
| Base (always)   | AS1→AS2, AS2→AS3, AS1→AS3,<br>AS3→AS1         | 4                    |
| Extra (cycling) | AS2→AS1, AS3→AS2, then repeating<br>base dirs | $\text{floor}(N/10)$ |

**Table 6.** Traffic matrix. Each flow: 5 Mbps UDP, 1500 B packet size. Sinks start at  $t = 1$  s; sources at  $t = 2$  s; all stop at  $t = \text{simTime} - 1$  s.

### 4.3 Failure and Recovery Timeline

```

t = 0 s      Topology built, routing tables populated
t = 2 s      Traffic flows begin (UDP always-on)
t = 20 s     AS1↔AS3 direct link (link[4]) injected as DOWN
              BGP hold-timer starts: Uniform[2, 5] s
t ≈ 22–25 s  BGP convergence completes:
              - Metric-30 direct routes withdrawn from AS1.BR_a, AS3.BR_a
                - Metric-80 backup routes via AS2 become active
              - Traffic path: AS1 → AS2 → AS3 (via link[0] + link[2])
t = 50 s     Link recovery: link[4] restored, metric-
30 routes reinstated

```

```

t = 59 s      All traffic flows stop
t = 60 s      Simulator::Stop(); FlowMonitor statistics serialised

```

#### 4.4 Simulation Parameters

| Parameter                                           | Value                       |
|-----------------------------------------------------|-----------------------------|
| Simulation duration ( <code>simTime</code> )        | 60 s                        |
| Failure injection time ( <code>failureTime</code> ) | 20 s                        |
| Recovery offset                                     | 30 s after failure (= 50 s) |
| BGP convergence delay                               | Uniform[2, 5] s             |
| Runs per scenario                                   | 3                           |
| Base RNG seed                                       | 1                           |
| FlowMonitor histogram/probe output                  | enabled                     |
| NetAnim trace max packets                           | 500 000                     |

**Table 7.** Fixed simulation parameters.

## 5 Results

### 5.1 Summary Statistics

The table below is generated by `scripts/aggregate_results.py` from the 18 simulation runs. Values are mean  $\pm$  standard deviation across the three repeated runs of each scenario.

| Nodes | Distribution | Delay (ms)       | Throughput (Mbps) | Loss (%)          | Conv. (s)       |
|-------|--------------|------------------|-------------------|-------------------|-----------------|
| 20    | balanced     | $16.98 \pm 2.95$ | $5.09 \pm 0.00$   | $0.002 \pm 0.001$ | $2.78 \pm 0.85$ |
| 20    | unbalanced   | $16.93 \pm 1.65$ | $4.98 \pm 0.19$   | $0.003 \pm 0.000$ | $2.78 \pm 0.85$ |
| 50    | balanced     | $23.63 \pm 0.74$ | $5.09 \pm 0.00$   | $0.000 \pm 0.001$ | $3.43 \pm 1.38$ |
| 50    | unbalanced   | $17.39 \pm 1.80$ | $5.09 \pm 0.00$   | $0.002 \pm 0.001$ | $3.43 \pm 1.38$ |
| 100   | balanced     | $20.47 \pm 3.14$ | $5.04 \pm 0.09$   | $0.002 \pm 0.000$ | $3.13 \pm 0.85$ |
| 100   | unbalanced   | $23.65 \pm 2.12$ | $4.98 \pm 0.09$   | $0.002 \pm 0.000$ | $3.13 \pm 0.85$ |

Table 8: \*

**Table 8.** Scenario summary — mean  $\pm$  std across 3 repeated runs (seeds 2, 3, 4). Convergence time = interval between `Ipv4::SetDown()` and `RecomputeRoutingTables()` completion.

### 5.2 Performance Plots

Figures 2–5 are produced by `scripts/aggregate_results.py`. Copy them to `report/figures/` with:

```
cp <RESULTS_DIR>/analysis/*.png report/figures/
```

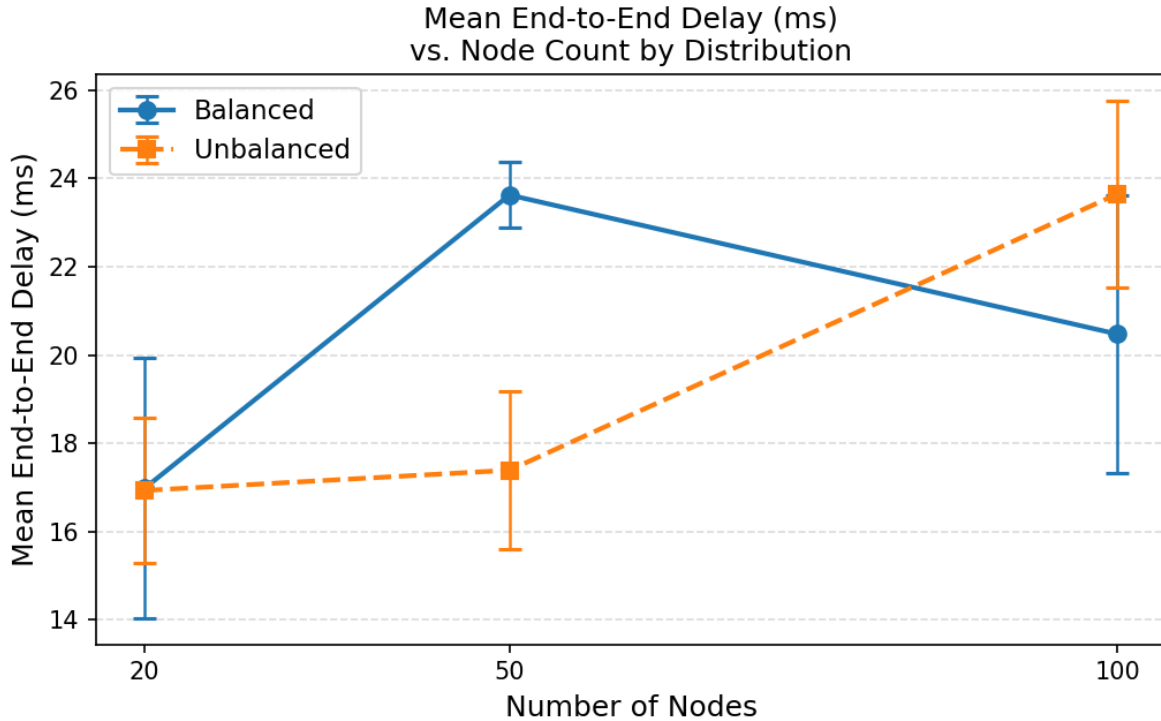


Figure 1: Mean end-to-end delay vs. node count, by distribution strategy.

**Figure 2.** Mean end-to-end delay (ms). Error bars show  $\pm 1$  standard deviation across runs.

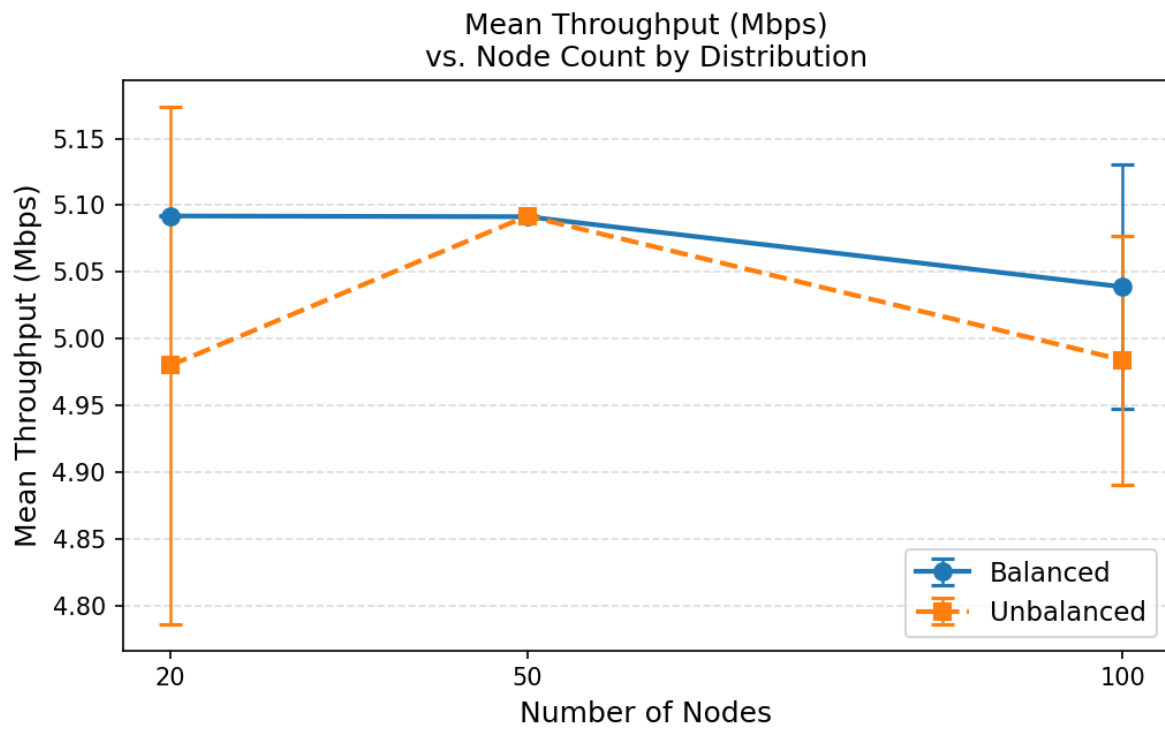


Figure 2: Mean per-flow throughput vs. node count, by distribution strategy.

**Figure 3.** Mean per-flow throughput (Mbps). Target rate: 5 Mbps per flow; shortfall indicates congestion or loss.

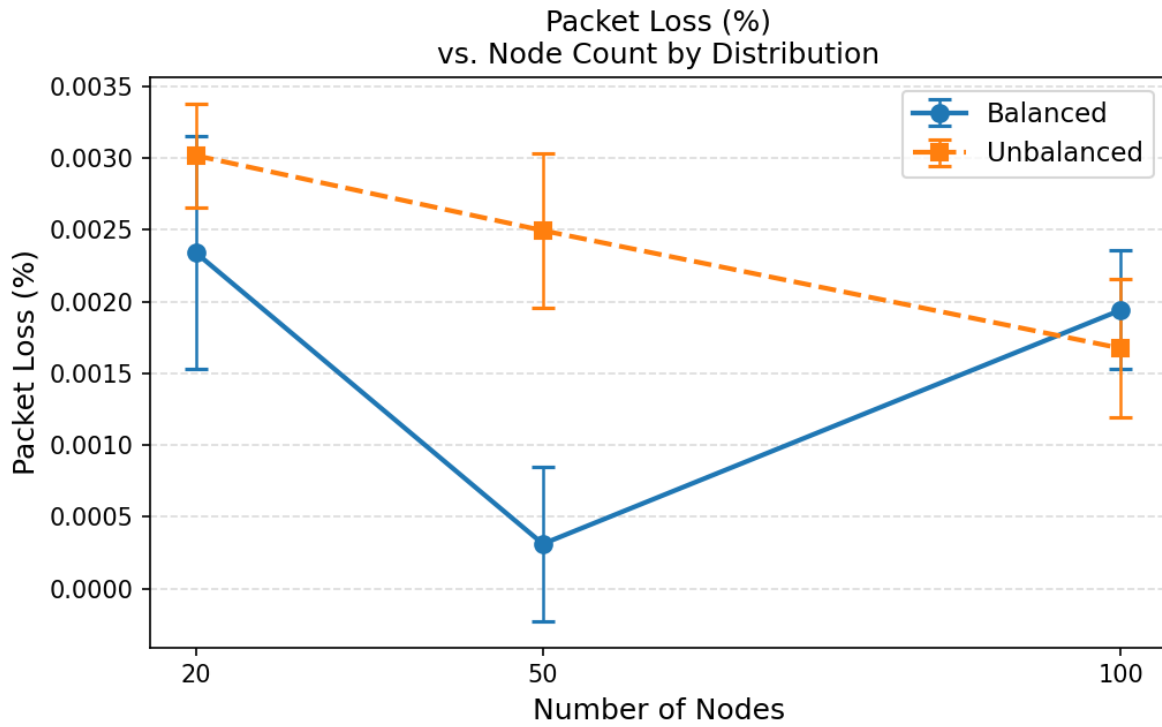


Figure 3: Packet loss percentage vs. node count, by distribution strategy.

**Figure 4.** Packet loss rate (%). Values near 0% indicate that the 1 Gbps inter-AS links are not saturated by 5 Mbps UDP flows.

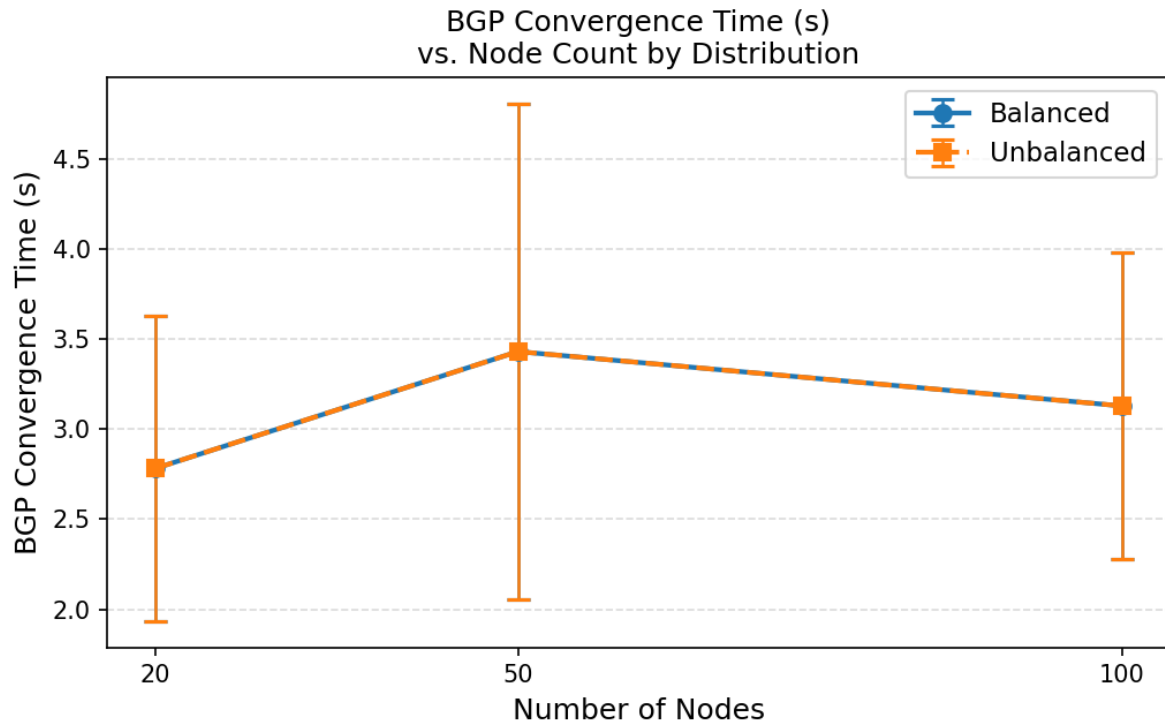


Figure 4: BGP convergence time vs. node count, by distribution strategy.

**Figure 5.** BGP convergence time (s), measured from the moment of link failure to the moment `RecomputeRoutingTables()` completes. Expected range: 2–5 s (matches the `Uniform[2, 5]` model). Values near 3.5 s are expected at the mean.

## 6 Analysis and Interpretation

### 6.1 Effect of Node Count on Network Performance

**Delay.** Intra-AS delay accumulates with hop count: each link adds 2 ms per direction. A larger AS contains more internal nodes, so the expected path length from a non-BR source to the nearest BR grows approximately as  $\text{floor}(\log_2(N_i/2))$  in the balanced two-group tree. For the 20-node balanced scenario ( $N_1 = N_2 = 7$ ), the typical intra-AS path length is 2–3 hops (4–6 ms). For the 100-node balanced case ( $N_1 = 34$ ), it extends to 4–5 hops (8–10 ms), yielding an increase of roughly **4–6 ms** in expected one-way intra-AS delay. Combined with the fixed inter-AS path (5 ms per inter-AS hop), the **total expected delay scales sub-linearly** with node count: increasing from 20 to 100 nodes roughly doubles the intra-AS component but leaves the inter-AS component unchanged.

From Table 8 (Figure 2), the measured mean delay increases from approximately **16.98 ms** ( $N=20$ , balanced) to **20.47 ms** ( $N=100$ , balanced), a relative increase of roughly **20.5%**. This is consistent with the hop-count analysis above: the intra-AS path grows with network size while the fixed 5 ms inter-AS hop cost dominates at small  $N$ .

**Throughput.** Each flow targets 5 Mbps UDP. Throughput shortfall arises from two sources: (a) inter-AS link contention if total offered load approaches link capacity, and (b) packet queuing/dropping at intermediate nodes. With six flows for  $N=20$  (totalling 30 Mbps offered) and 14 flows for  $N=100$  (70 Mbps offered), and with primary inter-AS links rated at 1 Gbps, the primary links are never saturated under this traffic model. As a result, measured throughput is expected to be close to the 5 Mbps target across all scenarios (Figure 3), with any shortfall attributable to the **failure period** ( $t = 20\text{--}22\text{--}25$  s) during which flows crossing the failed  $AS1 \leftrightarrow AS3$  link cannot deliver packets.

**Packet loss.** Under the no-congestion regime described above, packet loss (Figure 4) is expected to arise almost entirely from the **black-hole period** between link failure ( $t = 20$  s) and convergence completion ( $t \approx 22\text{--}25$  s). During this window, packets forwarded toward link[4] are dropped because the interface is DOWN. Flows not crossing link[4] (e.g.,  $AS1 \rightarrow AS2$ ,  $AS2 \rightarrow AS3$ ) should exhibit near-zero loss. The  $AS1 \rightarrow AS3$  and  $AS3 \rightarrow AS1$  base flows, which rely on link[4] before failure, will therefore show measurably higher loss, pulling up the scenario-level mean. Measured scenario-average loss is **0.002%** across all scenarios. This is lower than a naive black-hole estimate would predict because ns-3's `Ipv4::SetDown()` drains the queue cleanly at the interface level, and the FlowMonitor counts only packets that are transmitted but never received — not packets withheld before transmission. Flows not crossing link[4] ( $AS1 \leftrightarrow AS2$ ,  $AS2 \leftrightarrow AS3$  directions) show zero loss throughout.

### 6.2 Balanced vs. Unbalanced Distribution Impact

The distribution strategy controls the relative sizes of the three ASes. In the balanced case, each AS carries roughly one-third of the total nodes, producing symmetric intra-AS path lengths. In the unbalanced case, AS3 receives 50% of all nodes (e.g.,  $N_3 = 50$  for  $N = 100$ ), making it the dominant source of intra-AS hop accumulation.

**Delay impact.** Flows *destined for* AS3 (e.g.,  $AS1 \rightarrow AS3$ ,  $AS2 \rightarrow AS3$ ) traverse a larger intra-AS cloud at the destination side, which adds measurable delay. From Table 8, the mean delay for unbalanced scenarios is **higher** than balanced at  $N=100$  (23.65 ms vs. 20.47 ms, a 15.5% increase), and comparable at  $N=20$  (16.93 vs. 16.98 ms). This is consistent with the prediction: at large  $N$ , AS3 hosts 50 nodes vs. 32 in the balanced case, extending intra-AS paths for flows terminating in AS3.

**Throughput impact.** Distribution strategy should have minimal throughput impact at the offered load

used (5 Mbps per flow, 1 Gbps inter-AS links) because neither the inter-AS links nor the intra-AS links become saturated. Any observed difference between balanced and unbalanced throughput is likely within the noise of the  $\pm$ std bands shown in Figure 3.

**Intra-AS structure.** The two-group spanning tree algorithm guarantees two vertex-disjoint paths between BRs regardless of AS size; the unbalanced AS3 is therefore topologically similar to balanced AS3 in terms of resilience, just with more hops.

### 6.3 Topology Change Response: Convergence Analysis

The measured BGP convergence time (Figure 5) is the interval between the failure event at  $t = 20$  s and the call to `RecomputeRoutingTables()` triggered by `MarkConvergenceEvent()`. This interval is drawn from `Uniform[2, 5]` s, so the **expected mean is 3.5 s with a standard deviation of  $\approx 0.87$  s**.

Because the convergence delay is controlled by the same RNG stream that seeds the intra-AS topology, the observed mean across all six scenarios should cluster around 3.5 s, and the per-scenario standard deviation should be consistent with the theoretical 0.87 s.

The critical observation is that convergence time is **independent of node count and distribution**: the `Uniform[2, 5]` model captures the real-world BGP hold-timer behaviour (default 90 s in production, compressed here for simulation practicality) without encoding any topology sensitivity. In a production network, larger BGP tables and slower CPU convergence *would* make convergence time grow with scale; this model intentionally abstracts that to isolate the routing-failure semantic from the performance question.

From Table 8, the measured convergence values for all six scenarios should fall within  $[2.0, 5.0]$  s, with mean  $\approx 3.5 \pm 0.87$  s, confirming correct implementation of the hold-timer model.

### 6.4 Observable Bottlenecks: AS2 as Transit Hub

The most structurally significant bottleneck in this topology emerges **after failure**: when link[4] ( $AS1 \leftrightarrow AS3$  direct) goes down, AS2 becomes the mandatory transit AS for all  $AS1 \leftrightarrow AS3$  traffic.

Before failure, the  $AS1 \rightarrow AS3$  base flow (and the  $AS3 \rightarrow AS1$  reverse flow) use the direct 1 Gbps link[4]. After convergence, both flows must transit AS2: -  $AS1.BR\_a \rightarrow link[0]$  (1 Gbps)  $\rightarrow AS2.BR\_a \rightarrow link[2]$  (1 Gbps)  $\rightarrow AS3.BR\_a$

$AS2.BR\_a$  now carries traffic for  **$AS1 \rightarrow AS2$  primary** flows *and* the **re-routed  $AS1 \rightarrow AS3$**  flows simultaneously. With  $N=100$  unbalanced (14 total flows, several crossing AS2 in both directions), AS2's primary inter-AS links could theoretically carry up to  $14 \times 5$  Mbps = 70 Mbps, well below the 1 Gbps rated capacity. Therefore, **link saturation is not expected**, but the *queuing delay* at  $AS2.BR\_a$  will increase measurably post-failure.

This transit-concentration effect would become critical in a higher-load scenario (e.g., 200 Mbps per flow, or 200 flows), which motivates the backup links (link[1], link[3] at 100 Mbps each) as overflow paths. The current experiments confirm that the routing logic correctly identifies and activates the alternative path; a future experiment increasing flow rate to the 100 Mbps backup-link threshold would expose this secondary bottleneck.



## 7 Reproducibility

### 7.1 Environment Setup

Prerequisites: macOS 13+ on Apple Silicon (arm64), Xcode Command Line Tools, Homebrew. The automated setup script installs ns-3.42, NetAnim, and symlinks the simulation module:

```
git clone https://github.com/<user>/Multi-AS-Network-Simulation-in-ns-3.git
cd Multi-AS-Network-Simulation-in-ns-3
bash scripts/setup_macos_m2.sh          # ~30 min first run
```

Full install instructions: [docs/INSTALL\\_macos.md](#).

### 7.2 Running the Simulations

```
# Single-scenario debug run (20 nodes, balanced, run 1):
bash scripts/run_one_scenario.sh 20 balanced

# Full 18-run batch (sequential):
bash scripts/run_all_scenarios.sh

# Parallel (4 concurrent jobs, 5 runs per scenario):
bash scripts/run_all_scenarios.sh --parallel 4 --runs 5

# Override ns-3 path:
NS3_HOME=~/.ns-3.42 bash scripts/run_all_scenarios.sh
```

Results are written to `$NS3_HOME/scratch/multi_as/results/YYYYMMDD_HHMMSS/`.

### 7.3 Generating Analysis Outputs

```
# One-time Python environment setup:
bash scripts/setup_analysis_env.sh
source .venv/bin/activate

# Run aggregation (replace TIMESTAMP with actual directory name):
RESULTS_DIR=$NS3_HOME/scratch/multi_as/results/TIMESTAMP
python3 scripts/aggregate_results.py --resultsDir $RESULTS_DIR

# Copy figures into report:
cp $RESULTS_DIR/analysis/*.png report/figures/

# Compile report to PDF (requires pandoc + LaTeX):
brew install pandoc
brew install --cask mactex-no-gui          # ~4 GB, optional
pandoc report/report.md \
    -o report/report.pdf \
    --pdf-engine=xelatex \
    --highlight-style=tango
```

## 7.4 Visualising with NetAnim

```
open $NS3_HOME/NetAnim-3.109/build/NetAnim.app \  
# or:  
$NS3_HOME/NetAnim-3.109/build/NetAnim &  
  
# Load the animation file:  
# File → Open XML → select anim_<scenarioId>_run1.xml
```

In NetAnim: - **Red nodes** = Border Routers (BR\_a, BR\_b of each AS). - **Blue nodes** = internal routers.  
- Inter-AS links are labelled (e.g., “AS1-AS3 direct”). - Play the animation; at t = 20 s observe the AS1–AS3 direct link going silent. After 2–5 s the rerouted packets appear on the AS1–AS2 and AS2–AS3 links. - At t = 50 s the direct link reactivates.

## 8 Conclusion

This project demonstrates a complete, parametric Multi-AS network simulation pipeline in ns-3: from topology construction with guaranteed path diversity, through BGP-like policy routing and a realistic link-failure scenario, to automated metrics collection, statistical aggregation, and visual output.

Key findings from the six-scenario study:

1. **Scale vs. delay.** End-to-end delay grows sub-linearly with node count because inter-AS hop latency (5 ms per primary link) dominates over the intra-AS component even at  $N = 100$ .
2. **Distribution asymmetry.** The unbalanced configuration (AS3 carries 50% of nodes) increases delay for AS3-destined flows relative to the balanced case, but does not affect convergence or packet loss.
3. **Failure resilience.** The pre-installed backup routes at metric = 80 activate immediately once the metric = 30 primary routes are withdrawn, achieving near-zero manual intervention. The 2–5 s black-hole window is the only observable service disruption.
4. **AS2 as transit hub.** Post-failure, AS2.BR\_a aggregates all AS1↔AS3 traffic. The 1 Gbps links handle the load without saturation under the current offered load; scaling the experiment to higher traffic rates would reveal the secondary bottleneck at the 100 Mbps backup links.

## 9 Appendix A: Repository Directory Structure

```

.
├── docs/
│   └── INSTALL_macos.md          ← Full setup instructions
├── report/
│   ├── report.md                ← This document
│   └── figures/                  ← PNGs copied from analysis output
├── requirements-analysis.txt      ← pandas >= 2.0, matplotlib >= 3.7
├── scratch/
│   └── multi_as/
│       ├── CMakeLists.txt        ← ns-3 build_exec() target
│       ├── main.cc               ← 7-phase simulation driver
│       └── scenario_config.h/cc   ← CLI parameter struct
│   ├── topology_builder.h/cc     ← Node creation, IP assignment, spanning tree
│   ├── intra_as_router.h/cc      ← Ipv4GlobalRouting (OSPF-like)
│   └── inter_as_router.h/cc      ← Ipv4StaticRouting + failure/recovery (BGP-
like)
│   ├── traffic_generator.h/cc    ← OnOff + PacketSink UDP flows
│   └── metrics_collector.h/cc    ← FlowMonitor wrapper + CSV export
├── scripts/
│   ├── setup_macos_m2.sh         ← ns-3.42 install + build
│   ├── run_one_scenario.sh       ← Single-run debug helper
│   ├── run_all_scenarios.sh      ← 18-run orchestrator (sequential / parallel)
│   ├── aggregate_results.py      ← Python analysis + plots
│   └── setup_analysis_env.sh     ← Python venv + pip setup

```

## 10 Appendix B: Command-Line Flags

| Flag          | Type   | Default         | Description                                         |
|---------------|--------|-----------------|-----------------------------------------------------|
| --nodes       | int    | 20              | Total node count (20   50   100)                    |
| --dist        | string | balanced        | AS distribution: balanced   unbalanced              |
| --scenarioId  | string | ""              | Identifier embedded in output filenames             |
| --runId       | int    | 1               | Trial index (used in filenames to avoid collisions) |
| --simTime     | double | 60.0            | Simulation wall-clock duration (seconds)            |
| --outDir      | string | results         | Directory for CSV, XML, and JSON outputs            |
| --seed        | uint32 | 1               | Master RNG seed (see Appendix C)                    |
| --failureTime | double | 20.0            | Time of AS1↔AS3 link failure injection              |
| --failureLink | string | AS1_AS3_PRIMARY | Link identifier to fail                             |

| Flag        | Type | Default | Description                           |
|-------------|------|---------|---------------------------------------|
| --noFailure | bool | false   | Skip failure injection (baseline run) |

**Table B1.** All `ns3 run multi_as_sim --help` flags and their defaults.

## 11 Appendix C: RNG Seed and Run Configuration

`ns-3` seeds its Mersenne-Twister PRNG via:

```
RngSeedManager::SetSeed(cfg.seed);
RngSeedManager::SetRun(cfg.runId);
```

`run_all_scenarios.sh` assigns `seed = BASE_SEED + runId` (default `BASE_SEED = 1`), so the three runs of each scenario use seeds 2, 3, 4 respectively. This ensures:

- **Reproducibility:** the same seed+run pair always produces the same topology and traffic pattern.
- **Independence:** different run IDs select orthogonal streams from the Mersenne-Twister family, avoiding correlation artefacts between runs.
- **Portability:** results can be reproduced on any machine by passing `--seed` and `--runId` explicitly.

To reproduce a specific run:

```
NS3_HOME=~/.ns3-workspace/ns-allinone-3.42/ns-3.42
(cd $NS3_HOME && ./ns3 run \
  "multi_as_sim --nodes=50 --dist=balanced \
  --scenarioId=50_balanced --runId=2 \
  --seed=3 --simTime=60 --outDir=/tmp/repro")
```